



ЛЕКЦІЯ №6

СПИСКИ

Списки, кортежі,
методи над
списками

СПИСКИ І КОРТЕЖІ

Список

`list` - послідовність елементів, розташованих у певному порядку. Доступ до елементів здійснюється за індексом. Значення елементів списку можна змінювати.

Кортеж

`tuple` - послідовність будь-яких елементів. Кортеж аналогічний списку зі значеннями, які не змінюються.

ОСНОВНІ ВІДМІННОСТІ МІЖ LIST ТА TUPLE

Характеристика	List (Список)	Tuple (Кортеж)
Синтаксис	Квадратні дужки: [1, 2, 3]	Круглі дужки: (1, 2, 3)
Змінність (Mutability)	Mutable (можна змінювати, додавати чи видаляти елементи)	Immutable (незмінний після створення)
Пам'ять	Займає більше пам'яті	Займає менше пам'яті
Швидкодія	Працює повільніше	Працює швидше
Використання	Для колекцій даних одного типу, які можуть змінюватися	Для фіксованих структур даних (наприклад, координати (x, y))
Методи	Багато методів (append(), remove(), pop() тощо)	Мало методів (лише count() та index())

ПРИКЛАД

```
# List — можна змінювати
```

```
my_list = [1, 2, 3]
```

```
my_list[0] = 99          # Працює: [99, 2, 3]
```

```
my_list.append(4)        # Працює: [99, 2, 3, 4]
```

```
# Tuple — не можна змінювати
```

```
my_tuple = (1, 2, 3)
```

```
my_tuple[0] = 99         # Помилка: TypeError
```

СТВОРЕННЯ СПИСКІВ

У мові Python список позначається квадратними дужками [], а елементи списку розділяються комами. Наприклад, так створюється список з назвами моделей автомобілів:

```
>>> cars = ['Alfa Romeo', 'Volvo', 'Lamborghini', 'BMW',  
'Toyota']  
>>> print(cars)  
['Alfa Romeo', 'Volvo', 'Lamborghini', 'BMW', 'Toyota']  
>>> empty_list = []  
>>> another_empty_list = list()  
>>> a=[1, 12, 25, 100]
```

Функція **len()** повертає кількість елементів списку.

ПЕРЕТВОРЕННЯ ТИПІВ: ФУНКЦІЯ **LIST()**

Функція **list()** перетворює інші типи даних у список.

```
>>> list('sun')  
['s', 'u', 'n']
```

Щоб перетворити рядок у список можна також скористатися функцією **split()**, вказавши рядок-розділювач:

```
>>> mybirthday = '3/4/1981'  
>>> mybirthday.split('/')  
['3', '4', '1981']
```

ПЕРЕТВОРЕННЯ СПИСКУ В РЯДОК

Для перетворення списку у рядок, використовується функція `join()`.

```
>>> planets = ['Mercury', 'Jupiter', 'Earth', 'Mars',  
'Venus']  
>>> ', '.join(planets)  
'Mercury, Jupiter, Earth, Mars, Venus'
```

Перетворення числового списку

```
a=[1,2,3,4]  
b=list(map(str,a))  
s=' '.join(b)  
print(s)  
1 2 3 4
```

ВВЕДЕННЯ СПИСКУ З РЯДКА ЗНАЧЕНЬ

```
x= input().split()  
a=list(map(int,x))  
print(x)  
print (a)
```

Функція `map()` застосовує вказану функцію (наприклад, `int()`, `float()`, `str()` тощо) до кожного елемента з послідовності (списка, кортежа, множини тощо). Отриманий результат можна перетворити в `list()`, `tuple()`, `set()` тощо.

```
PS C:\Users\User> & C:/Users/  
1 5 28 3 4  
['1', '5', '28', '3', '4']  
[1, 5, 28, 3, 4]
```

ДОСТУП ДО ЕЛЕМЕНТІВ СПИСКУ

Списки є впорядкованими наборами даних, тому для доступу до будь-якого елементу списку слід повідомити Python позицію (**індекс**) потрібного елемента.

Індекси приймають тільки цілочисельні значення. Щоб звернутися до елементу у списку, вкажіть ім'я списку, за яким слідує індекс елемента в квадратних дужках.

```
>>> cars = ['Alfa Romeo', 'Volvo', 'Lamborghini', 'BMW',  
'Toyota']  
>>> print(cars[1])  
Volvo  
>>> print(cars[-1])  
Toyota
```

ІНДЕКС ЕЛЕМЕНТА

Якщо необхідно дізнатися індекс елемента у списку за його значенням, використовується функція **index()**.

```
>>> planets = ['Mercury', 'Jupiter', 'Earth', 'Mars',  
'Venus']  
>>> planets.index('Earth')  
2
```

ЗМІНА ЕЛЕМЕНТІВ СПИСКУ

На відміну від рядків список можна змінювати:

```
>>> planets = ['Mercury', 'Jupiter', 'Earth', 'Mars',  
               'Venus']  
>>> planets[2] = 'Saturn'  
>>> planets  
['Mercury', 'Jupiter', 'Saturn', 'Mars', 'Venus']
```

ЗРІЗИ

Використовуючи розділення **(зріз)** списку, можна витягти зі списку підпоследовність, що також буде списком:

```
>>> planets = ['Mercury', 'Jupiter', 'Earth', 'Mars', 'Venus']
>>> planets[0:3]
['Mercury', 'Jupiter', 'Earth']
>>> planets[::2]
['Mercury', 'Earth', 'Venus']
>>> planets[::-3]
['Venus', 'Jupiter']
>>> planets[::-1] # розміщення елементів списку у зворотному
порядку
['Venus', 'Mars', 'Earth', 'Jupiter', 'Mercury']
```

МЕТОДИ

Дії над об'єктами зручно виконувати за допомогою функцій передаючи їм об'єкти як аргумент.

Метод - це по суті функція, яка "закріплена" за певним об'єктом і яка виконує певні дії над цим об'єктом.

Записується це наступним чином: об'єкт чи посилання на нього, далі ставиться крапка, і після крапки метод об'єкта:

`object.method()`

Пам'ятаючи що метод — це функція, його можна викликати, що й означають круглі дужки після імені метода.

```
s.split() # викликаємо метод split об'єкту типу str
```

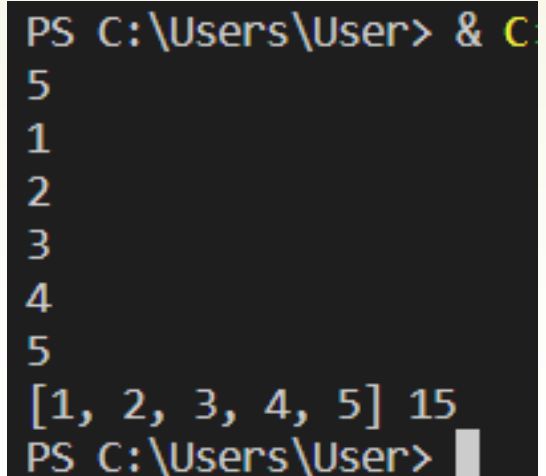
Таблиця "Методи списків"

Метод	Призначення
<code>list.append(x)</code>	Додає елемент <code>x</code> в кінець списку <code>list</code>
<code>list.extend(L)</code>	Розширює список <code>list</code> , додаючи у кінець всі елементи списку <code>L</code>
<code>list.insert(i, x)</code>	Вставляє у списку <code>list</code> на <code>i</code> -ий елемент значення <code>x</code>
<code>list.remove(x)</code>	Видаляє перший елемент у списку <code>list</code> , який має значення <code>x</code> ; <code>ValueError</code> , якщо такого елемента не існує
<code>list.pop(i)</code>	Видаляє із списку <code>list</code> <code>i</code> -ий елемент і повертає його; якщо індекс не вказано, видаляється останній елемент
<code>list.index(x, start, end)</code>	Повертає положення у списку <code>list</code> першого елемента зі значенням <code>x</code> (при цьому пошук ведеться від <code>start</code> до <code>end</code>)
<code>list.count(x)</code>	Повертає кількість елементів із списку <code>list</code> зі значенням <code>x</code>
<code>list.sort(reverse=False)</code>	Сортує елементи списку <code>list</code> ; якщо необов'язковий параметр <code>reverse</code> має значення <code>True</code> , список сортується в зворотному порядку
<code>list.reverse()</code>	Розвертає список <code>list</code>
<code>list.copy()</code>	Поверхнева копія списку <code>list</code>
<code>list.clear()</code>	Очищає список <code>list</code>

ПОРЯДКОВЕ ВВЕДЕННЯ СПИСКУ

```
#введення масиву з клавіатури
n=int(input())
a=[]
for i in range(n):
    a.append(int(input()))
print(a)
```

```
#введення масиву з клавіатури та обчислення суми
a,s=[],0
n=int(input())
for i in range(n):
    x=int(input())
    a.append(x)
    s+=x
print(a,s)
```



```
PS C:\Users\User> & C:\Users\User\python3.exe C:\Users\User\script.py
5
1
2
3
4
5
[1, 2, 3, 4, 5] 15
PS C:\Users\User>
```

СОРТУВАННЯ

```
#method sort True - спадання False - зростання  
a=[1, 5, 3, 4, 5]  
a.sort(reverse=True)  
b=sorted(a)  
print(a,b)
```

```
PS C:\Users\User> & C:/Users/User/A  
[5, 5, 4, 3, 1] [1, 3, 4, 5, 5]
```

СТАТИСТИЧНІ ДАНІ

```
>>> digits = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
>>> min(digits)
0
>>> max(digits)
9
>>> sum(digits)
45
```

ПРОХІД ПО СПИСКУ

Перехід по списку

```
thislist = ["apple", "banana", "cherry"]  
for x in thislist:  
    print(x)
```

Перебір індексних номерів

```
thislist = ["apple", "banana", "cherry"]  
for i in range(len(thislist)):  
    print(thislist[i])
```

РОЗУМІННЯ СПИСКУ

Розуміння списків пропонує коротший синтаксис, коли потрібно створити новий список на основі значень існуючого списку.

```
newlist = [expression for item in iterable if condition == True]
```

На основі списку фруктів вам потрібен новий список, який міститиме лише фрукти з літерою «a» в назві.

```
fruits = ["apple", "banana",  
"cherry", "kiwi", "mango"]  
newlist = []  
  
for x in fruits:  
    if "a" in x:  
        newlist.append(x)  
  
print(newlist)
```

```
fruits = ["apple", "banana", "cherry",  
"kiwi", "mango"]  
  
newlist = [x for x in fruits if "a" in x]  
  
print(newlist)
```

МАТРИЦІ

	0	1	2	3	4	5	6
0	0	2	4	1	4	6	9
1	3	3	4	5	77	5	8
2	5	0	3	2	6	-1	0
3	6	4	5	45	-2	8	19
4	5	7	1	1	1	0	12

```
matrix = [  
    [0, 2, 4, 1, 4, 6, 9],  
    [3, 3, 4, 5, 77, 5, 8],  
    [5, 0, 3, 2, 6, -1, 0],  
    [6, 4, 5, 45, -2, 8, 19],  
    [5, 7, 1, 1, 1, 0, 12]  
]
```

```
# Доступ до елемента (рядок 1, стовпчик 4):  
print(matrix[1][4]) # Виведе: 77
```

ПРОХІД ПО МАТРИЦІ

1. Прохід за елементами (порядково)

Найпростіший спосіб, якщо вам не потрібні номери рядків і стовпчиків:

```
for row in matrix:
    for elem in row:
        print(elem, end=" ")
    print()    # Перехід на новий рядок
```

2. Прохід за індексами і та j

Використовується, коли потрібно знати точні координати елемента або змінювати значення безпосередньо в матриці:

```
rows = len(matrix)          # Кількість рядків (5)
cols = len(matrix[0])       # Кількість стовпчиків (7)

for i in range(rows):
    for j in range(cols):
        print(f"matrix[{i}][{j}] = {matrix[i][j]}")
```